

UNIT - 3

Pushdown Automata:- Using this one we can say that any language is CFL (or) not.

- It is more powerful than FSM
- FSM has averely limited memory but PDA has more memory

→ PDA = Finite state machine + A stack

Formal Definition of PDA:-

→ A pushdown Automata is formally defined by 7 Tuples as shown below:

$P = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$ where,

Q = A finite set of states

Σ = A finite set of Input symbol

Γ = A finite stack Alphabet

δ = The Transition Function

q_0 = The start state

z_0 = The start stack symbol

F = Final state (or) Accepting state.

- δ takes as argument a triple $\delta(q, a, x)$ where
- $\rightarrow q$ is a state in Q
 - $\rightarrow a$ is either input symbol ($a \in \Sigma$)
 - $\rightarrow x$ is a stack symbol, that is member of Γ
 - $\rightarrow$ The output of δ is finite set of pairs (p, Y) where
 - p is a new state
 - Y is string of stack symbol that replaces x at the top of the stack.

eg:- If $Y = \epsilon$ then stack is popped
 If $Y = x$ then stack is unchanged
 If $Y = yax$ then x is replaced by z and a is pushed onto the stack.

- $\rightarrow$ PDA divided into 2 types. (1) DPDA (2) NPDA
- $\rightarrow$ DPDA (Deterministic PDA) & NPDA (non-deterministic PDA) both are same except δ :

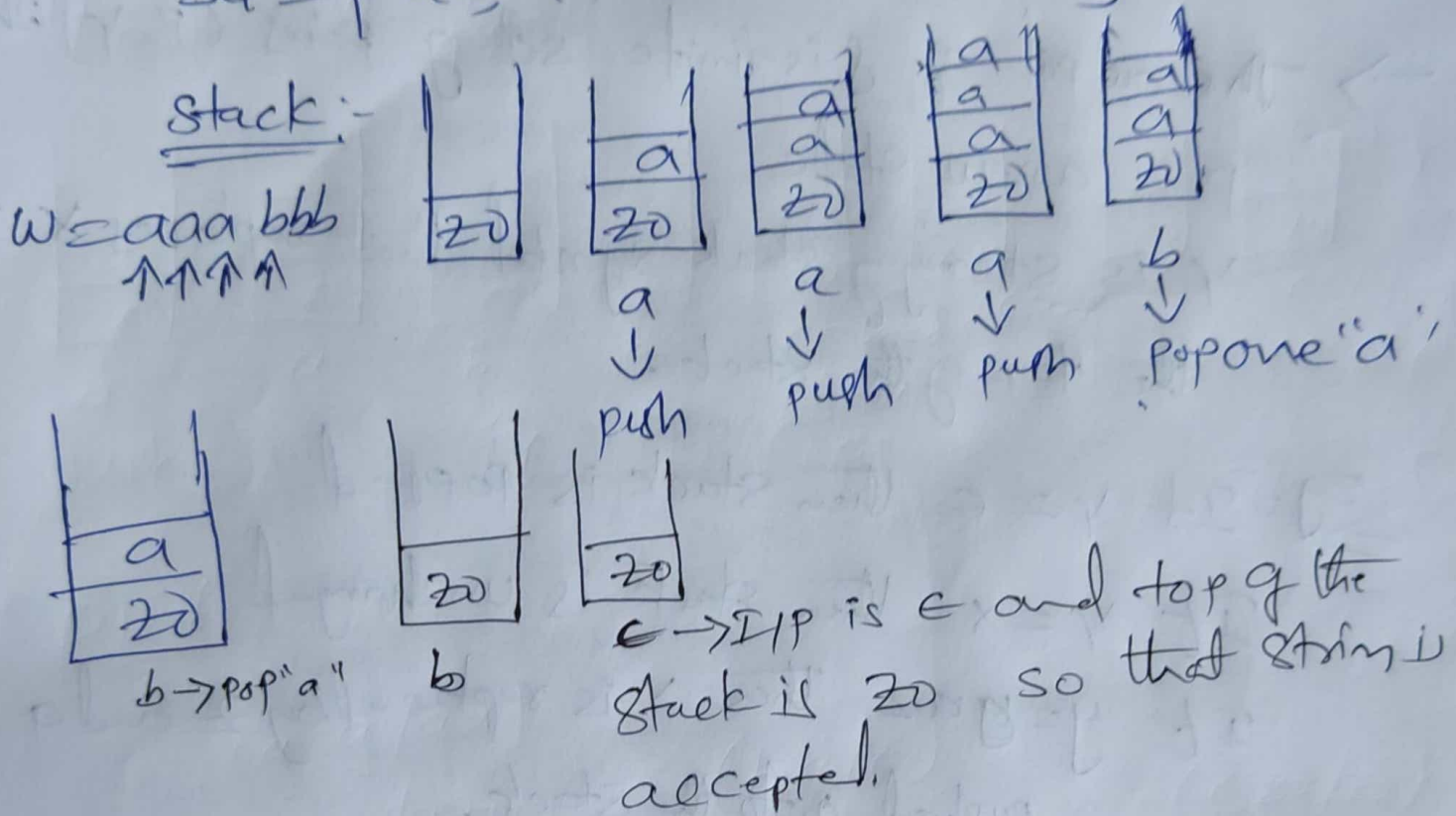
DPDA: $\delta: Q \times \{\Sigma \cup \epsilon\} \times \Gamma^* \rightarrow Q \times \Gamma^*$

NPDA: $\delta: Q \times \{\Sigma \cup \epsilon\} \times \Gamma^* \rightarrow 2^Q \times \Gamma^*$

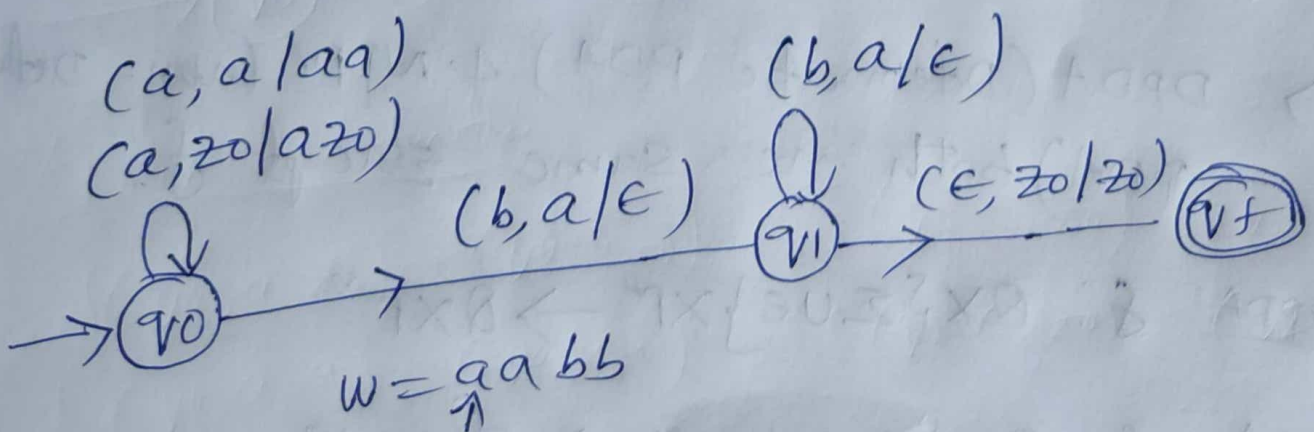
Ex 1: Construct a PDA that accepts the language
 $= \{a^n b^m \mid n \geq 1\}$ where $\Sigma = \{a, b\}$

Sol: $L = \{a^n b^m \mid n \geq 1\}$

set = $\{ab, aabb, \dots\}$



8 Construction of PDA



Transition function

$$\delta(q_0, a, z_0) = (q_0, a z_0)$$

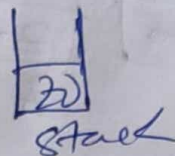
$$\delta(q_0, a, a) = (q_0, a a)$$

$$\delta(q_0, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, b, a) = (q_1, \epsilon)$$

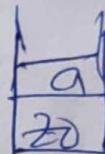
$$\delta(q_1, \epsilon, z_0) = (q_f, z_0)$$

If $w = a a b b$



$$\delta(q_0, a, z_0) = \delta(q_0, a z_0)$$

$$\delta(q_0, a, a) = (q_0, a a)$$



$$\delta(q_0, b, a) = (q_1, a z_0)$$

$$\delta(q_1, b, a) = (q_1, z_0)$$

$$\delta(q_1, \epsilon, z_0) = (q_f, z_0)$$

→ we reached to final state so given string is ~~not~~ accepted

If $w = a a a b b$

$$(q_0, a a a b b, z_0)$$



$$(q_0, a a b b, a z_0)$$

$$(q_0, a a b b, a a z_0)$$



$$(q_0, b b, a a a z_0)$$

$$(q_0, b, a a z_0)$$



$$(q_1, \epsilon, a a z_0) \rightarrow \text{not}$$

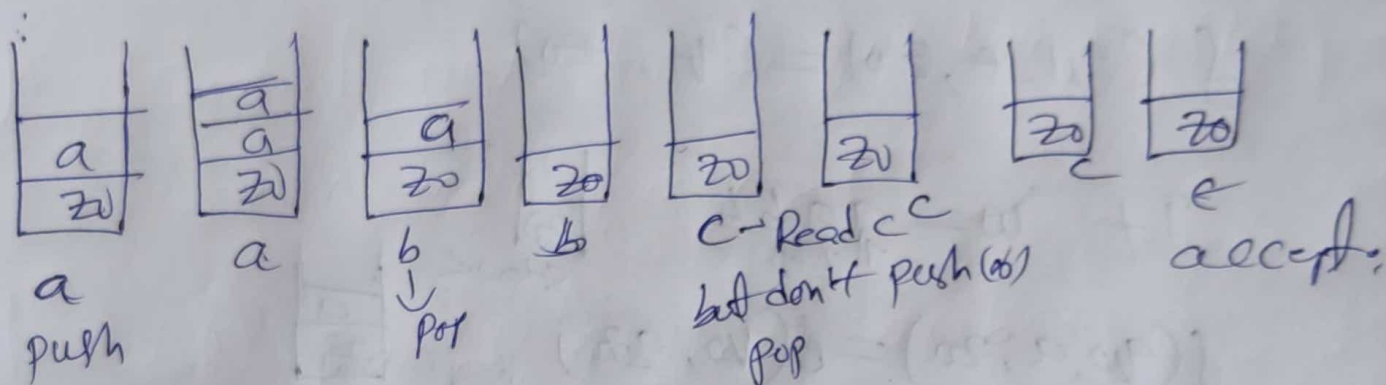
mentioning " q_1 " so not accepted as after reading i/p stack is not empty

eg 2: Construct PDA for $L = a^n b^n c^m / n, m \geq 1$

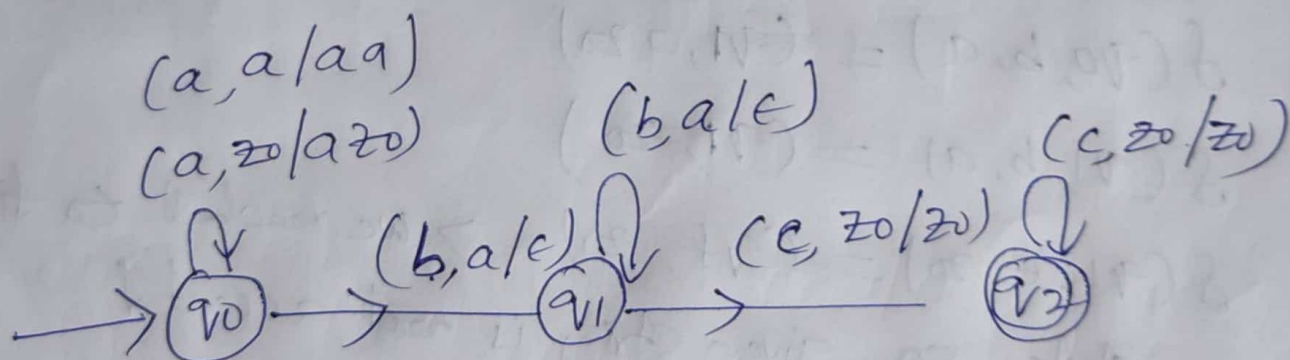
Sol: $L = a^n b^n c^m / n, m \geq 1$

$S = \{abc, aabbccc, aaabbbccc, \dots\}$

If $w = aabbccc$



DPDA:



Transition function:

$$\delta(q_0, a, z_0) = (q_0, a z_0)$$

$$\delta(q_0, a, a) = (q_0, a a)$$

$$\delta(q_0, b, a) = (q_1, e)$$

$$\delta(q_1, b, a) = (q_1, e)$$

$$\delta(q_1, c, z_0) = (q_2, z_0)$$

$$\delta(q_2, c, z_0) = (q_2, z_0)$$

If the string $w = aaabbbcc$

$(q_0, aaabbbcc, z_0)$

↓

$(q_0, aabbbcc, a z_0)$

↓

$(q_0, abbbcc, aa z_0)$

↓

$(q_0, bbbcc, aaa z_0)$

↓

$(q_1, bbcc, aaa z_0)$

↓

$(q_1, cc, aaa z_0)$

↓

$\delta(q_1, c, a) \rightarrow$ not mentioned so

given string is not accepted

eg 3: Construct PDA for given $L = a^n b^m c^n / n, m \geq 1$

sol: $S_L = \{abc, aabcc, aabbbcc, \dots\}$

→ Here using stack I will compare ~~and~~ a & c
so it is CFL & we can construct PDA

$w = aaabbbccc$

a
a
a
z_0

no ja 'a' you can push

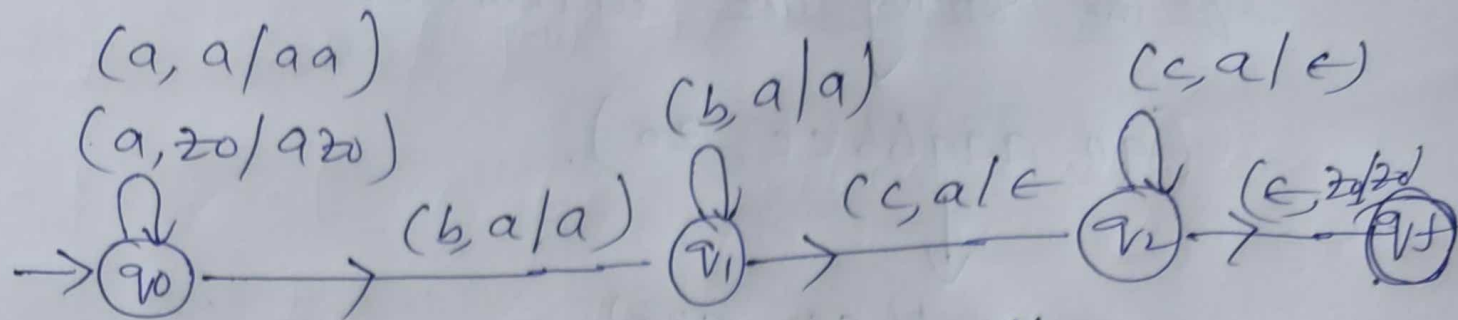
a
a
a
z_0

→ If we get 'b' leave it
don't push & pop

a
a
z_0

a
z_0

→ If we get c you can pop
If the stack ' z_0 ' it ended

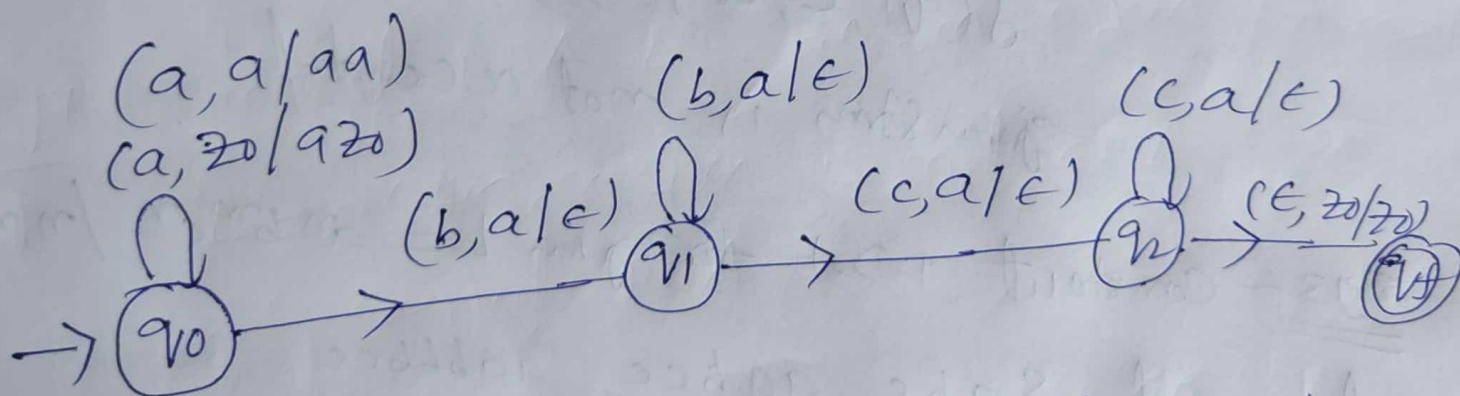


→ write the transition function yourself.

eg 4:- Construct PDA for the given Language (L)
 $= a^{m+n} b^m c^n \quad / m, n \geq 1$

sol:- $S_L = \{ aabb, aaabbc, aaabccc, \dots \}$

If $w = aaabcc$



→ write the transition function yourself. take
 $w = aaabcc$ trace it. your self.

eg 5: Construct PDA $L = a^n b^{m+n} c^m \mid n, m \geq 1$

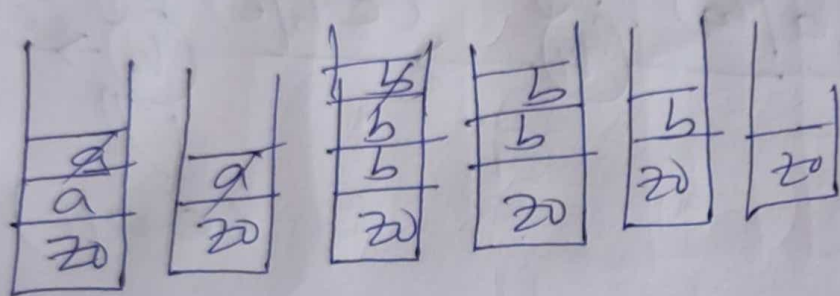
sol: $L = a^n b^{m+n} c^m$

$L = a^n b^n b^m c^m$, Here no 'a' & b are same
no 'b' & c are same

Set = { abbc, aabb bbb ccc ... }

It can accepted by stack.

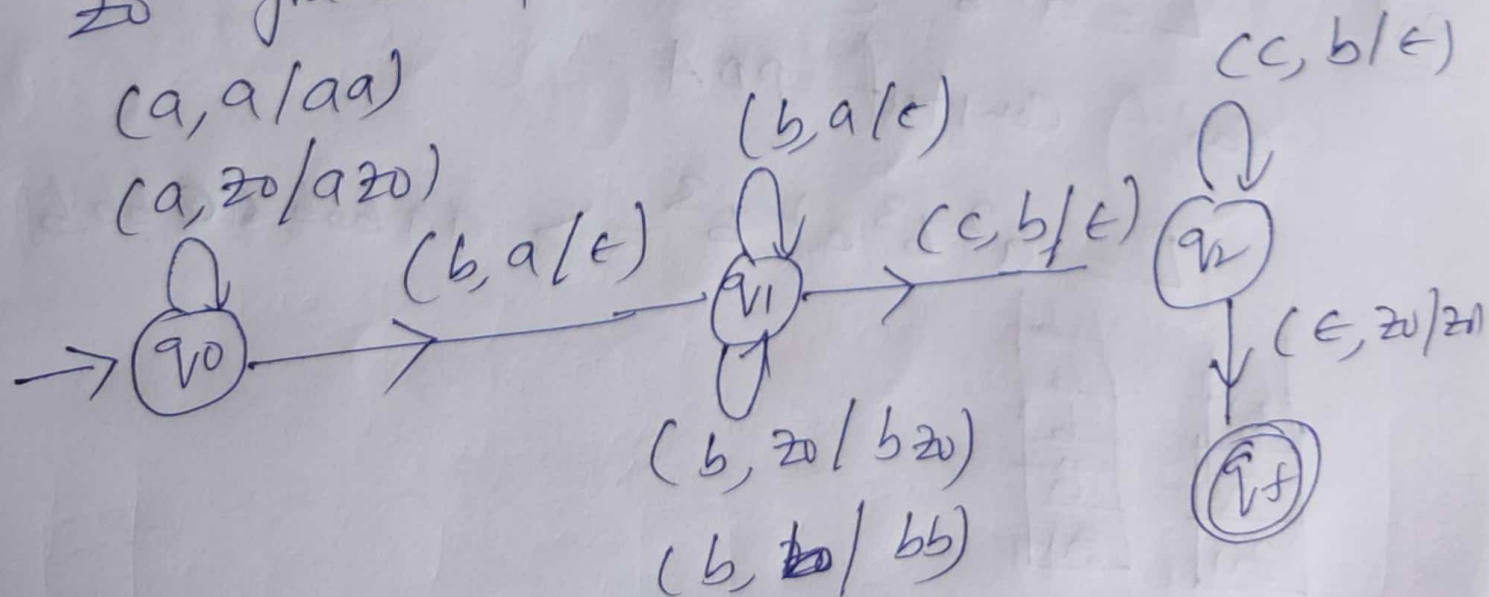
If $w = \underline{a} \underline{a} \underline{b} \underline{b} \underline{b} \underline{b} \underline{c} \underline{c} \underline{c}$



Case 1: push 'a' to stack

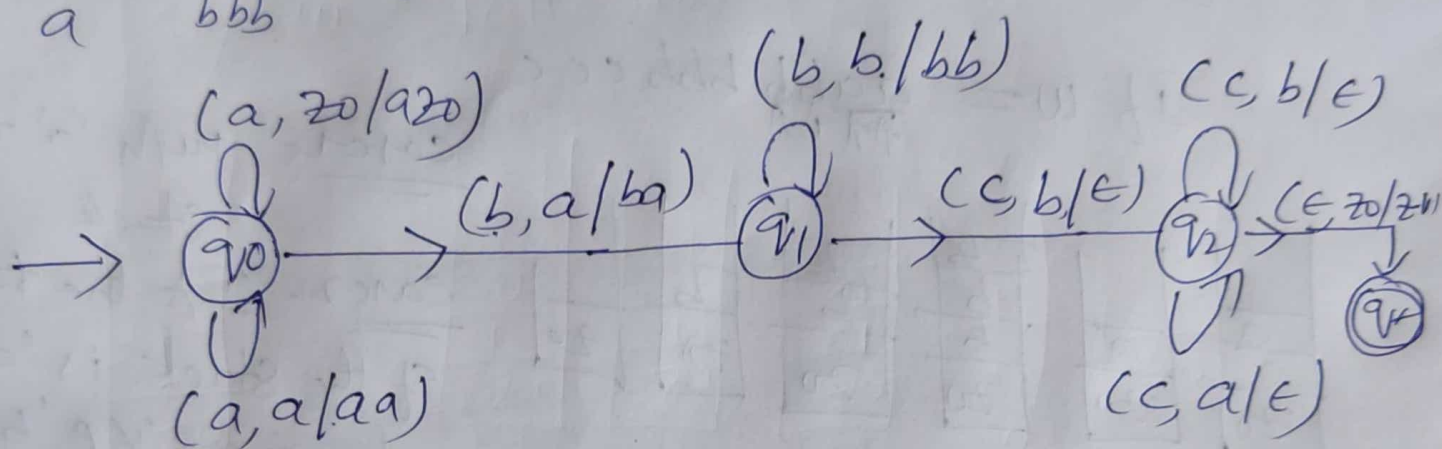
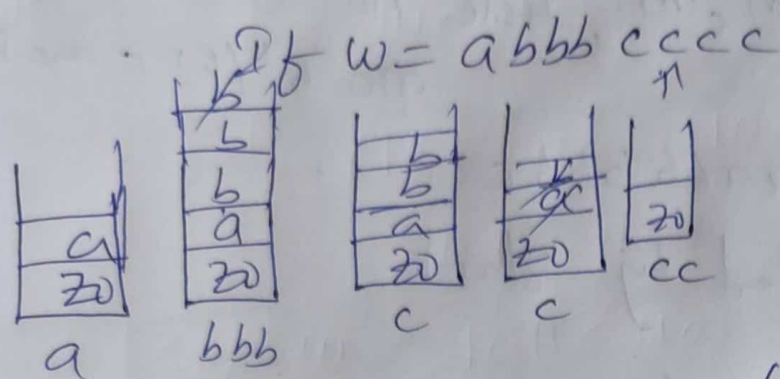
Case 2: If the top of the stack is 'a' for given i/p 'b' then you can pop

Case 3: If the i/p is 'b', the top the stack is 'ε' you can push the 'b'



eg 6 - Construct PDA for the $L = a^n b^m c^n m / n, m \geq 1$

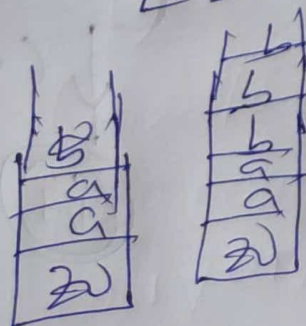
Sol:- $\text{set} = (abcc, aabccc, abbbcccc, \dots)$



eg 7 - Construct PDA $L = a^n b^m c^n d^m / n, m \geq 1$

Sol:- $L = a^n b^m c^n d^m$ is not CFL so we can't construct PDA

$L = a^2 b^3 c^2 d^3 \Rightarrow aabbbccdd$

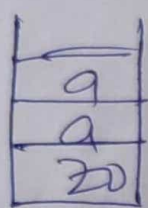


eg Construct PDA for $L = a^n b^{2n} / n \geq 1$

Sol: $L = a^n b^{2n} / n \geq 1$

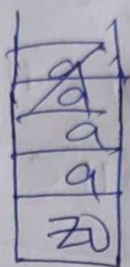
set = {abb, aabbbb, ...}

If $w = aabbbb$

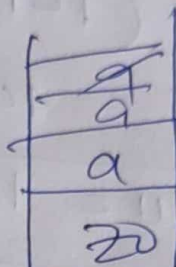


a

for one "a" you
push '2' a's



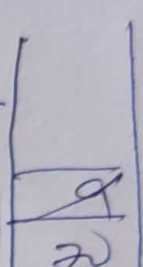
a



b



b



b



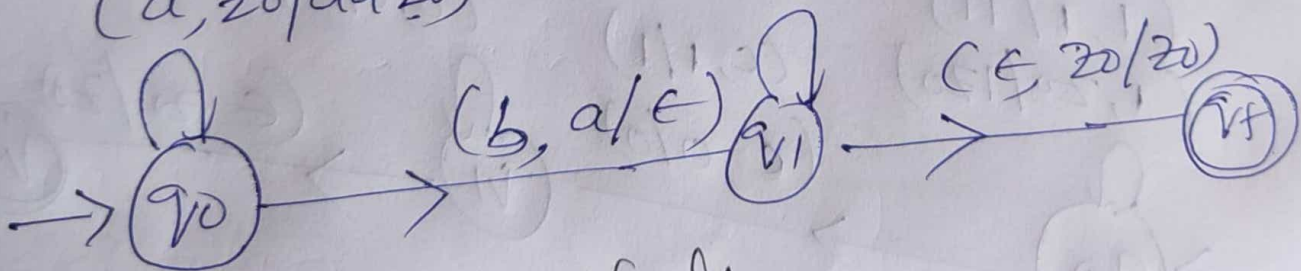
b

Top of the stack is "z0" so it can
accepted.

(a, a / aa a)

(a, z0 / aa z0)

(b, a / ε)



Transition function..

$(q_0, a, z_0) = (q_0, aa z_0)$

$(q_0, a, a) = (q_0, aaa)$

$(q_0, b, a) = (q_1, \epsilon)$

$(q_1, b, a) = (q_1, \epsilon)$

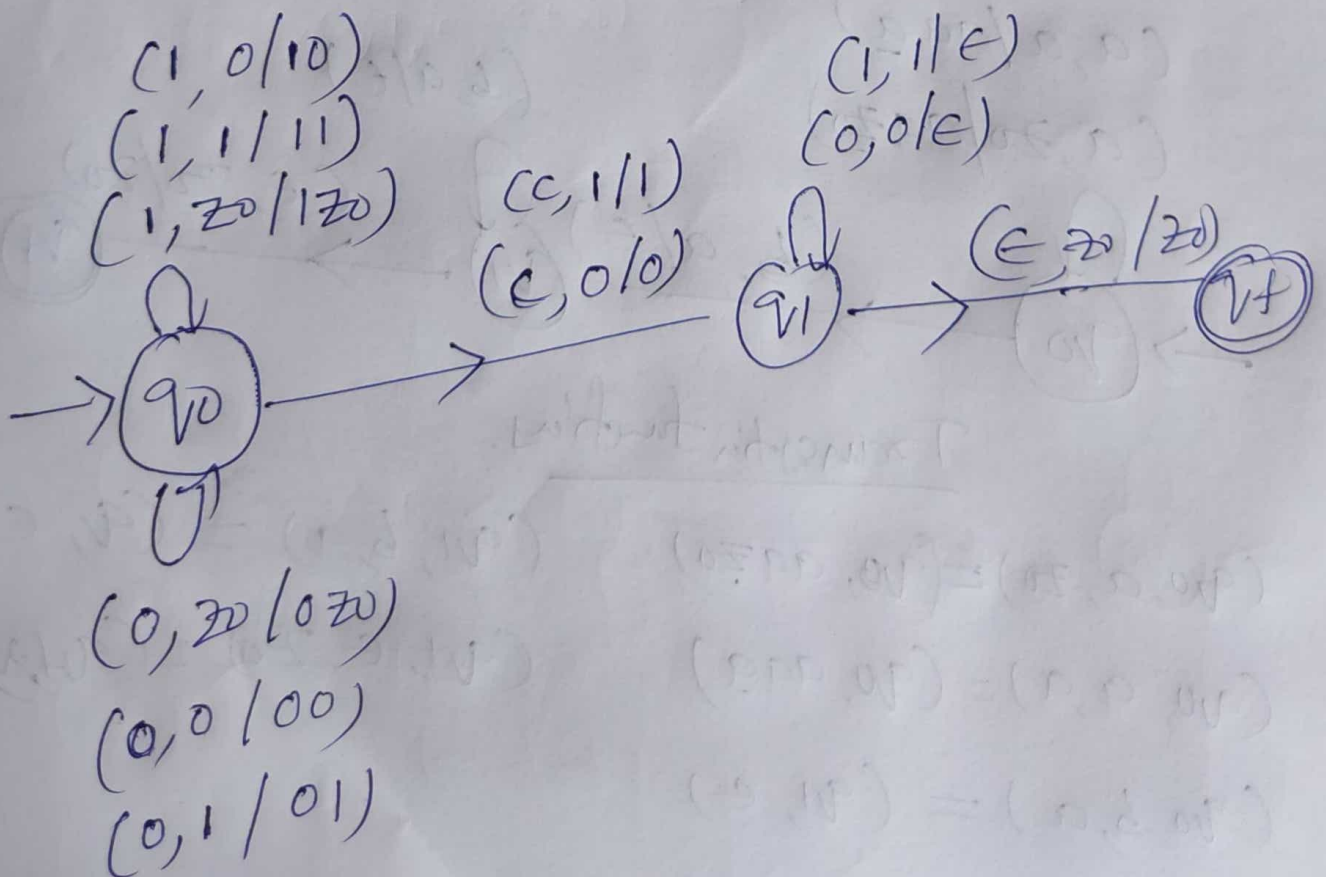
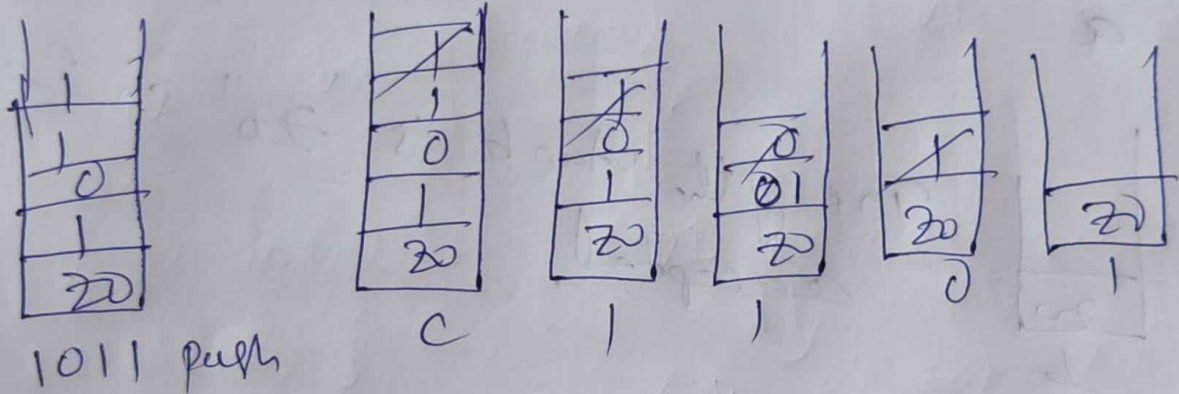
$(q_1, \epsilon, z_0) = (q_f, z_0)$

Ex 9.2 ^{N.P.D} Construct PDA for given $L = w c w^R /$
 $w \in (a, b)^+$

Sol. $L = w c w^R$

Set = $(1c1, 0c1, 10c01, 001c100, \dots$
 $1001c1101, \dots$

If $w = 1011c1101$



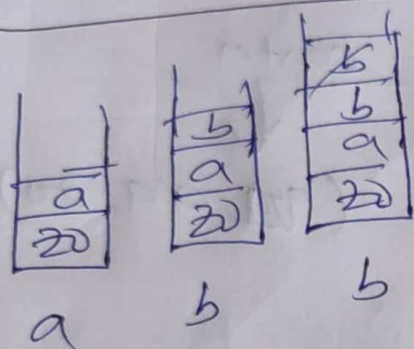
Example 2 NPDA

① Construct NPDA for the $L = \{w^R \mid w \in (a,b)^+\}$

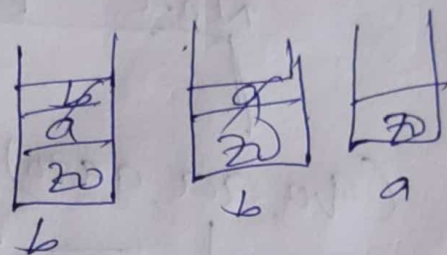
Sol: $L = \{w^R\}$

Set = $\{aa, bb, \underline{abba}, abbbba, \dots\}$

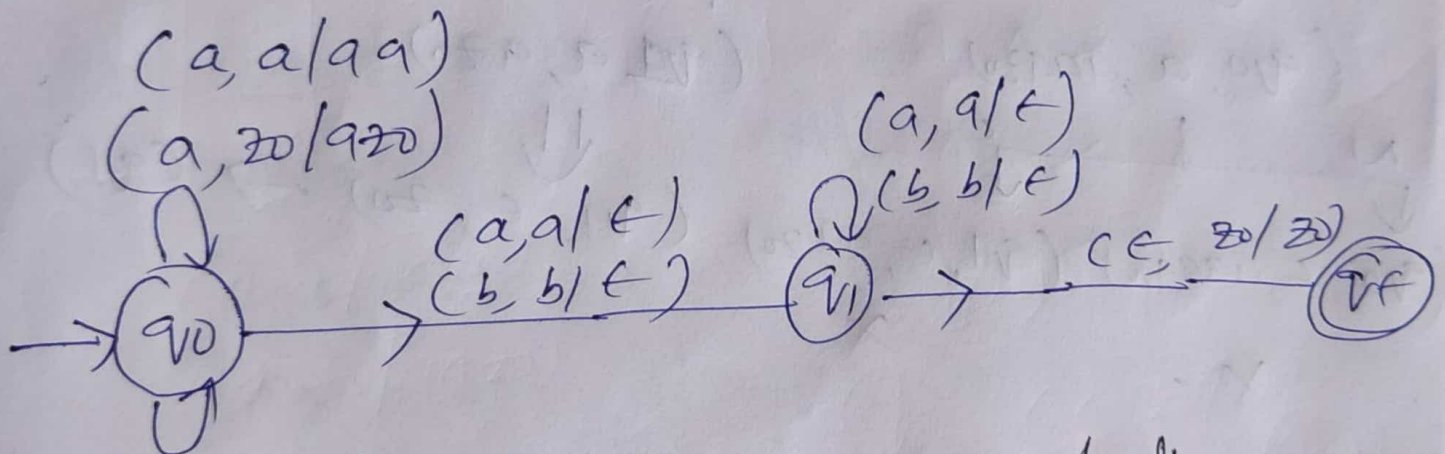
If the $w = abbbba$



I will assume abb is w



string is accepted



$(a, b/ab)$

$(b, z0/bz0)$

$(b, a/ba)$

$(b, b/bb)$

Transition function:

$(q_0, a, z_0) = (q_0, a z_0)$

$(q_0, a, a) = (q_0, aa) \text{ or } (q_1, \epsilon)$

$(q_0, b, z_0) = (q_0, b z_0)$

$(q_0, b, b) = (q_0, bb) \text{ or } (q_1, \epsilon)$

$(q_0, a, b) = (q_0, ab)$

$(q_0, b, a) = (q_0, ba)$

$$(q_1, a, a) = (q_1, e) \quad (q_1, e, zo) = (q_1, zo)$$

$$(q_1, b, b) = (q_1, e)$$

If $w = aaaa$

$$(q_0, aaaa, zo)$$

$$(q_0, aaa, a zo)$$

No-c center

$$(q_0, aa, a a zo)$$

$$(q_0, a, a a a zo)$$

$$(q_1, a, a zo)$$

$$(q_0, a, a a a zo)$$

$$(q_0, e, a a a a zo)$$

$$(q_1, e, a a zo)$$

$$(q_1, e, zo) \rightarrow (q_1)$$

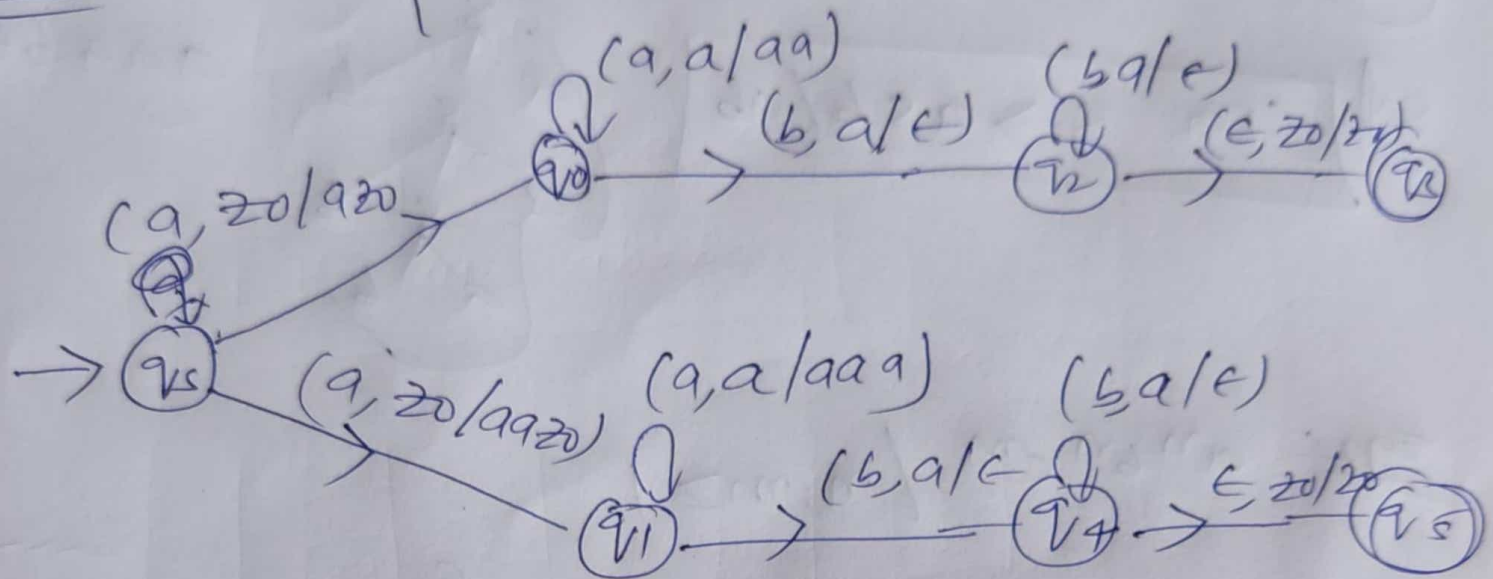
$$(q_0, e, a a a a zo)$$

X

X

Ex 2: Construct NPDA for $L = \{a^n b^n / n \geq 1\} \cup \{a^n b^{2n} / n \geq 1\}$

Sol: set = $\{ab, a^2b^2, a^3b^3, a^4b^4, \dots\}$



Write the transition function.

eg

$$\begin{aligned}
 (q_5, a, z_0) &= (q_0, a z_0) & (q_1, a a z_0) \\
 (q_0, a, a) &= (q_0, a a), & (q_0, b, a) = (q_2, \epsilon) \\
 (q_2, b, a) &= (q_2, \epsilon), & (q_2, \epsilon, z_0) = (q_3, z_0) \\
 (q_1, a, a) &= (q_1, a a a), & (q_1, b, a) = (q_4, \epsilon) \\
 (q_4, b, a) &= (q_4, \epsilon) & (q_4, \epsilon, z_0) = (q_5, z_0)
 \end{aligned}$$

Convert CFL to CFG

Eg1

$a^n b^n / n \geq 1$

Sol: CFG: $S \rightarrow asb / ab$



aabb

Eg2: $a^n b^n c^m / n, m \geq 1$

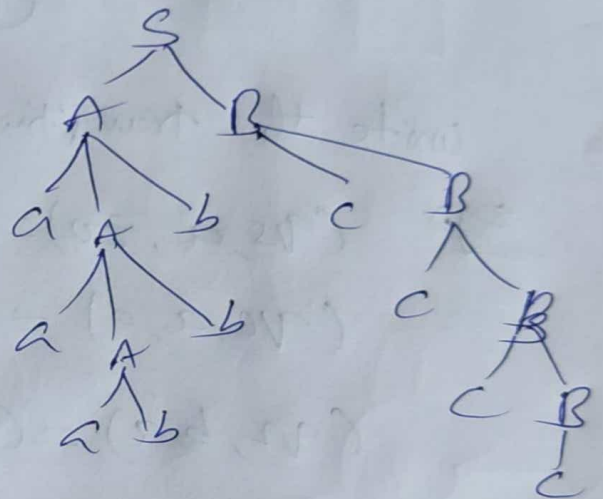
Sol

$L = \underline{a^n b^n c^m}$

$S \rightarrow AB$

$A \rightarrow aAb / ab$

$B \rightarrow cB / c$



Eg3: $a^n b^n c^m d^m / n, m \geq 1$

Sol

$L = \underline{a^n b^n c^m d^m}$

$S \rightarrow AB$

$A \rightarrow aAb / ab$

$B \rightarrow cBd / cd$

eg 4: $a^n b^{2n} / n \geq 1$

sol:-

$S \rightarrow asbb / abb$

Turing Machine

→ FA: The input string

a	b	b	a	ε
---	---	---	---	---

→ PDA: → The i/p string
→ Stack

→ Turing machine:

→ A Tape

B	a	a	a	ε	ε	ε	B
---	---	---	---	---	---	---	---

← Tape head →
↑ R/W

→ B - ~~Blank~~ Blank symbol.

Final Con

- operation on the Tape:

- Read / Scan symbol below the tape head.
- update / write a symbol below the tape head
- move the Tape Head one step LEFT
- move the Tape Head one step RIGHT.

→ Using Turing Machine the given Recursively enumerable language is accepted way

→ In Turing machine we will use Recursively Enumerable language & Unsatisfiable Grammar.

Turing Machine (Formal Definition)

→ A Turing machine can be defined as set of 7 tuples,
 $M = (Q, \Sigma, \Gamma, \delta, q_0, b, F)$

$Q \rightarrow$ Non-empty set of states.

$\Sigma \rightarrow$ non-empty set of symbols.

$\Gamma \rightarrow$ non-empty set of Tape symbol

$\delta \rightarrow$ Transition function defined as

$$Q \times \Sigma \rightarrow Q \times \Gamma \times (R/L)$$

$q_0 \rightarrow$ Initial state

$b \rightarrow$ Blank symbol

$F \rightarrow$ set of final states.

Transition function can be written as

$$\delta(q_0, a) \rightarrow (q_1, X, R)$$

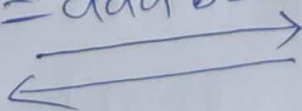
eg:- $L = a^n b^n, n \geq 1$ | Tape:

B	B	B	a	a	a	b	b	b	B
---	---	---	---	---	---	---	---	---	---

set = {ab, aabb, ...}

$w = aaaa bbb$

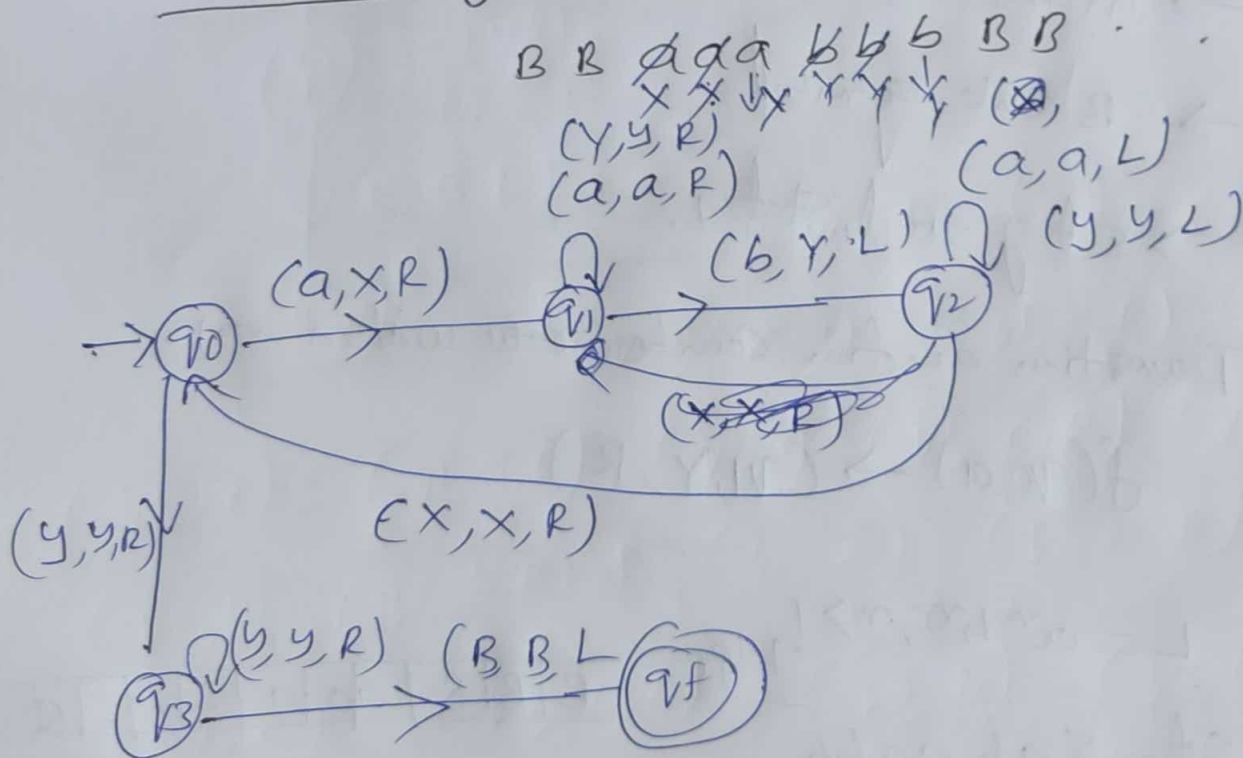
→



Algorithm-

- Change "a" to "x"
- Move Right to First "1", If it is not there Reject.
- After encounter "1" change to "y"
- Move Left to Left most "0"
- Repeat the above steps until no more "0's"
- Make sure no more "1's" remain.

Construction of Turing Machine



Transition Table

q _i	a	b	B	Y	X
q ₀ (Start)	(q ₁ , X, R)			(q ₃ , Y, R)	
q ₁	(q ₁ , a, R)	(q ₂ , Y, L)		(q ₁ , Y, R)	
q ₂	(q ₂ , a, L)			(q ₂ , Y, L)	(q ₀ , X, R)
q ₃			(q₃, B, L) (q ₃ , B, L)	(q ₃ , Y, R)	

If $w = \text{B a a b b B B}$

$\Rightarrow (q_0, \text{a a b b B B})$
 $\downarrow$

$(q_1, \text{X a b b B B}) \rightarrow (q_1, \text{X a b b B B})$
 $\downarrow$

$(q_2, \text{X a Y b B B})$
 $\downarrow$

$(q_2, \text{X a Y b B B})$
 $\downarrow$

$(q_1, \text{X X Y b B B}) \leftarrow (q_1, \text{X X Y b B B}) \leftarrow (q_0, \text{X a Y b B B})$
 $\downarrow$

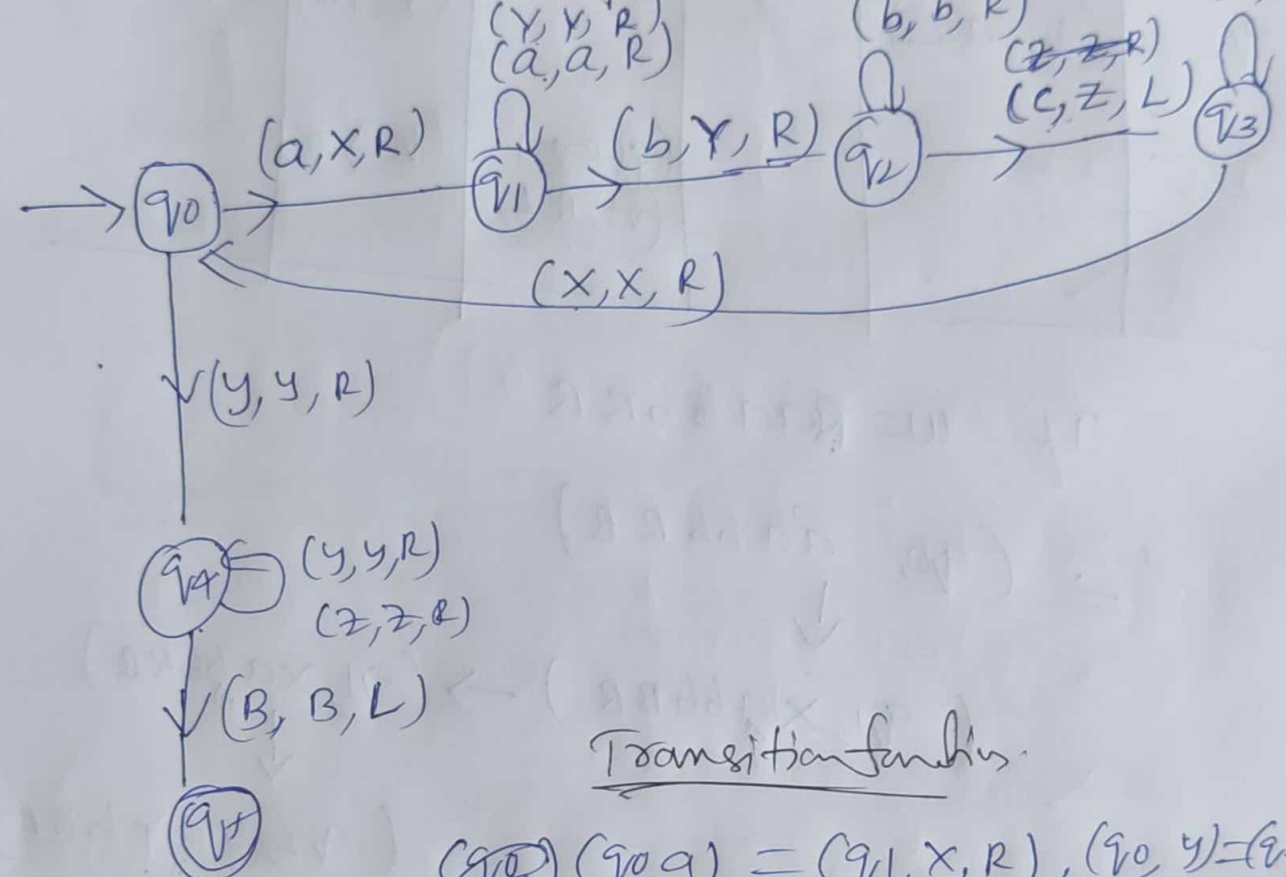
$(q_2, \text{X X Y Y B B}) \rightarrow (q_2, \text{X X Y Y B B}) \rightarrow (q_0, \text{X X Y Y B B})$
 $\downarrow$

$(q_1, \text{X X Y Y B B})$
 $\downarrow$

$(q_1, \text{X X Y Y B B})$
 $\downarrow$

eg 2: Construct the Turing machine for the
 $| = a^n b^n c^n \quad n \geq 1$

sol:
 set = ~~{abc}~~, {abc, aabbcc, ...} (z, z, L)
(a, a, L)
(y, y, L)
(b, b, L)
 $\Sigma \cap P =$ BB ^{xx} ^{yy} ^{zz} ~~aa~~ ~~bb~~ ~~cc~~ BB (z, z, R)
 (y, y, R)
 (a, a, R) (b, b, R) (z, z, R)
 (c, z, L)



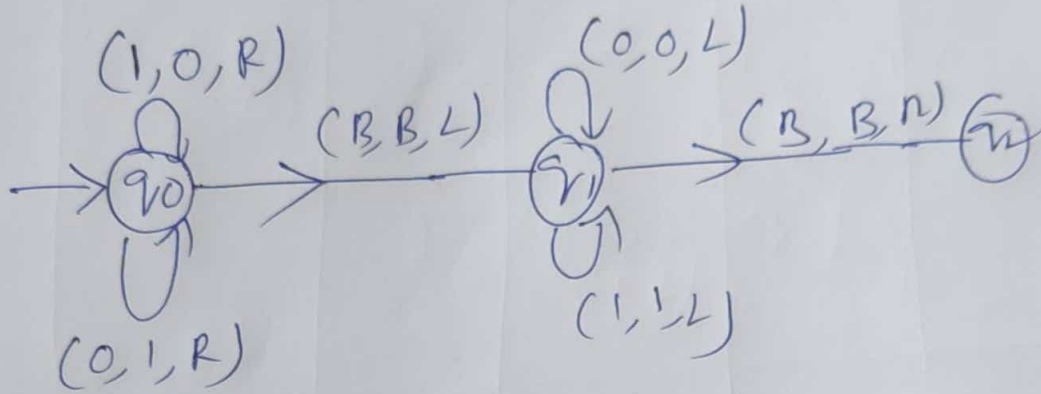
Transition function

- ~~(q0)~~ $(q_0, a) = (q_1, x, R), (q_0, y) = (q_4, y, R)$
 $(q_1, a) = (q_1, a, R), (q_1, b) = (q_2, y, R)$
 $(q_2, b) = (q_2, b, R), (q_2, z) = (q_2, z, R)$
 $(q_2, c) = (q_3, z, L), (q_3, b) = (q_3, b, L)$
 $(q_3, y) = (q_3, y, L), (q_3, a) = (q_3, a, L)$
 $(q_3, z) = (q_3, z, L), (q_4, y) = (q_4, y, R)$
 $(q_4, z) = (q_4, z, R), (q_4, B) = (q_5, B, L)$

Eg3: Construct the TM for 1's Complement of a Binary number.

Sol:-

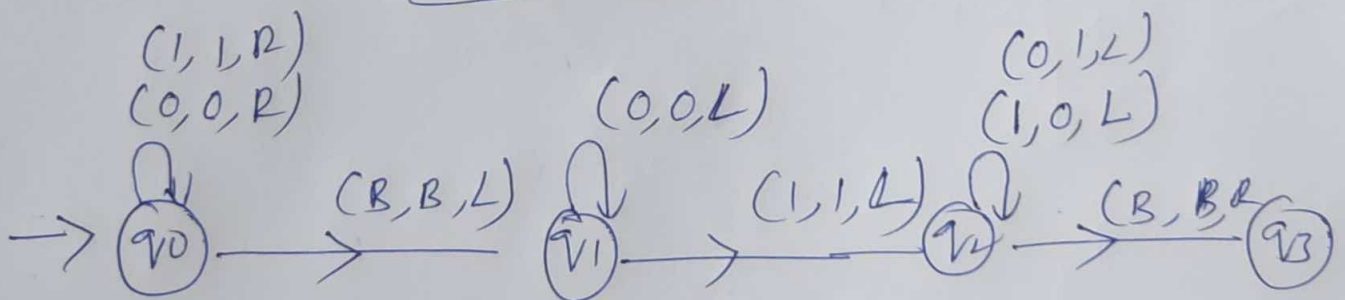
$W = \underline{B}10101BB$ 1's Complement = 01010



Eg4: Convert given Binary number into 2's Complement.

Sol:-

$VP = B0111000R \rightarrow$ 2's complement is
1001000 so you read from end.



Conversion of CFG to PDA

→ For converting given CFG to PDA, by this method the necessary condition is that the first symbol on R.H.S. production must be terminal symbol.

The rule that can be used to obtain PDA from CFG is

Rule 1: For non-terminal symbol, add following rule.

$$\delta(q, \epsilon, A) = (q, \alpha)$$

where the production rule is $A \rightarrow \alpha$.

Rule 2:- For each terminal symbol, add following rule.

$$\delta(q, a, a) = (q, \epsilon) \text{ for every terminal symbol}$$

Ex 1:- Construct PDA for the given CFG

$$S \rightarrow OBB$$

$$B \rightarrow OS / IS / O$$

Test whether 0104 is acceptable by this PDA.

Sol:-

Let $PDA = \{q\}, \{0, 1\}, \{S, B, \epsilon\}, \delta, q, \epsilon, F\}$

The production rules δ can be given as

$$R_1: \delta(q, \epsilon, S) = \{(q, OBB)\}$$

$$R_2: \delta(q, \epsilon, B) = \{(q, OS), (q, IS), (q, O)\}$$

$$R_3: \delta(q, 0, 0) = \{ (q, \epsilon) \}$$

$$R_4: \delta(q, 1, 1) = \{ (q, \epsilon) \}$$

Testing 0104i. 010000 against PDA

$$\delta(q, \epsilon, 010000, s) \vdash \delta(q, 0, 10000, 0BB) \quad R_1$$

$$\delta(q, 10000, \underline{B}B) \quad R_3$$

$$\delta(q, 10000, 1SB) \quad R_2$$

$$\delta(q, 0000, SB) \quad R_4$$

$$\delta(q, 0000, \underline{B}0BBB) \quad R_1$$

$$\delta(q, 000, BBB) \quad - R_3$$

$$\delta(q, \underline{0}00, 0SBB) \quad - R_2$$

$$\delta(q, 00, SBB)$$

$$\delta(q, 00, 0BBB)$$

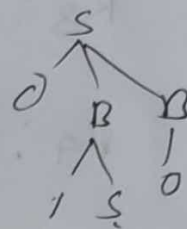
$$\delta(q, 0, BBB)$$

$$\delta(q, 0, 0SBB)$$

$$\delta(q, \epsilon, SBB)$$

$$R_3: \delta(q, 0, 0) = \{q, \epsilon\}$$

$$R_4: \delta(q, 1, 1) = \{q, \epsilon\}$$



Testing 0104 i.e. 010000 against PDA

$$\delta(q; 010000, s) \leftarrow \delta(q, 010000, 0BB) \quad R_1$$

$$\delta(q, 10000, BB) \quad R_3$$

$$\delta(q, 10000, 1SB) \quad R_2$$

$$\delta(q, 0000, SB) \quad R_4$$

$$\delta(q, 0000, 0BBB) \quad R_1$$

$$\delta(q, 000, BBB) \quad R_3$$

$$\delta(q, 000, 0BB) \quad R_2 \leftarrow (q, 0)$$

$$\delta(q, 00, BB) \quad R_3$$

$$\delta(q, 00, 0B) \quad R_2$$

$$\delta(q, 0, B) \quad R_3$$

$$\delta(q, 0, 0) \quad R_2$$

$$\delta(q, \epsilon) \quad R_3$$

string is accepted.

eg2:- Convert the grammar

$$S \rightarrow \cancel{0S1} \quad S \rightarrow 0A, A \rightarrow 0AB / 1 \\ B \rightarrow 1$$

Sol:- PDA = $(Q, \Sigma, \Gamma, \delta, q_0, F)$

$$= (q_0, q_1, q_f), (0, 1), (S, A, 0, 1), q_0, q_f)$$

→ the mapping function δ will be

$$\delta(q_0, \epsilon, S) = (q_1, 0A) \quad R_1$$

$$\delta(q_1, \epsilon, A) = (q_1, 0AB) \quad (or) \quad (q_1, 1) \quad R_2$$

$$\delta(q_1, \epsilon, B) = (q_1, 1) \quad R_3$$

$$\delta(q_1, 0, 0) = (q_1, \epsilon) \quad R_4$$

$$\delta(q_1, 1, 1) = (q_f, \epsilon) \quad R_5$$

$$\underline{w = 01}$$

$$\delta(q_1, 01, S) \leftarrow \delta(q_1, 01, 0A) \quad R_1$$

$$\delta(q_1, 1, A) \quad \cancel{R_2} \quad R_4$$

$$\delta(q_1, 1, 1) \quad R_2$$

$$\delta(q_f, \epsilon) \quad \text{---} R_5$$